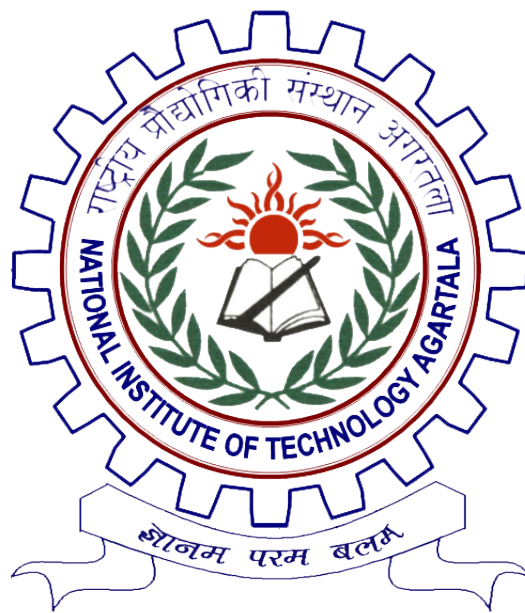


DIGITAL HORIZON CONNECTORS



Mainak Dasgupta (22UCS006)

**COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY, AGARTALA**

INDIA-799046

APRIL, 2026

DIGITAL HORIZON CONNECTORS

*Report submitted to
National Institute of Technology, Agartala
for the award of the degree
of
Bachelor of Technology*

*by
Mainak Dasgupta (22UCS006)*

*Under the Guidance of
Mr. Vivek Rodge
Software Development Manager, Amdocs*

*and
Dr. Awnish Kumar
Department Supervisor
Assistant Professor
CSE Department, NIT Agartala*



**COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY AGARTALA
APRIL, 2026**

Dedicated To

This report is dedicated to **Prof. Mrinal Kanti Debbarma**, HOD, CSE Department, NIT Agartala, my Project Manager **Mr. Vivek Rodge**, Software Development Manager, Amdocs and my Project Supervisor **Dr. Awnish Kumar**, Assistant Professor, CSE Department, NIT Agartala for sharing their valuable knowledge and encouragement and showing confidence on me all the time, to each of the faculties of the department who contributed to my development as a professional and helped me to achieve this goal, to my parents who supported me and believed in me and to all those people who have somehow contributed to the creation of this project.

“First, solve the problem. Then, write the code.”

- John Johnson

REPORT APPROVAL FOR B.TECH

This report entitled “***Digital Horizon Connectors***”, by **Mainak Dasgupta (22UCS006)** is approved in partial fulfilment of the requirements for the award of ***Bachelor of Technology*** in ***Computer Science & Engineering***.

Prof. Mrinal Kanti Debbarma
(Head of the Department)
Professor
Computer Science and Engineering Department
NIT Agartala

Dr. Awnish Kumar
(Department Supervisor)
Assistant Professor
Computer Science and Engineering Department
NIT Agartala

Mr. Vivek Rodge
(Project Manager)
Software Development Manager
Amdocs

Date:_____

Place: NIT, Agartala

DECLARATION

I declare that the work presented in this report titled “***Digital Horizon Connectors***”, submitted to the **Computer Science and Engineering Department, NIT Agartala**, for the award of the ***Bachelor of Technology*** degree in ***Computer Science & Engineering***, represents my ideas in my own words and where others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

APRIL, 2026
Agartala

Mainak Dasgupta
(22UCS006)

CERTIFICATE

It is certified that the work contained in the dissertation titled ‘***Digital Horizon Connectors***’, by **Mainak Dasgupta (22UCS006)** have been carried out under our supervision. This work has not been submitted elsewhere for a degree.

Mr. Vivek Rodge
(Project Manager)
Software Development Manager
Amdocs

The dissertation titled “***Digital Horizon Connectors***”, by **Mainak Dasgupta** bearing the Enrollment Number **22UCS006** is respectively endorsed for submission for the **Bachelor of Technology in Computer Science & Engineering Department, NIT Agartala**.

Dr. Awnish Kumar
Assistant Professor
(Department Supervisor)
Computer Science and Engineering Department
NIT Agartala

Acknowledgement

I take this opportunity to express my deep sense of gratitude to all who helped me directly or indirectly during this project work.

Firstly, I would like to thank my manager, **Mr. Vivek Rodge**, for being an exceptional guide and the best advisor I could ever have. His advice, encouragement and constructive feedback are sources of innovative ideas, inspiration and the causes behind the successful completion of this report. His confidence shown to me was the biggest source of inspiration for me. It has been a privilege working with him for the past 6 months.

I am highly obliged to all the faculty members of the Computer Science and Engineering Department for their support and encouragement. I also thank **Prof.(Dr.) S. K. Patra**, Director, NIT Agartala; **Prof. Mrinal Kanti Debbarma**, H.O.D, CSE Department; and **Dr. Awnish Kumar**, Assistant Professor, CSE Department, for providing excellent computing and other facilities, without which this work could not achieve its quality goal.

- Mainak Dasgupta (22UCS006)

List of Figures

2.1	Five-phase workflow lifecycle of the API Development Tool, showing the key artifacts passed between each phase.	9
4.1	Layered system architecture of the API Development Tool, showing internal subcomponents, communication protocols, and deployment boundaries.	23
4.2	Data flow through the utility layer: two parallel parser pipelines (Swagger and JAR) feed extracted field information into the interactive mapping stage, which produces the user-config YAML that drives final module generation.	29

List of Tables

1	Comparison of manual and tool-assisted connector development.	3
2	Feature comparison of existing approaches and the API Development Tool. . .	21
3	Consolidated findings matrix across evaluation dimensions.	42
4	Prioritised future work roadmap.	48

Abstract

Developing enterprise API connectors in telecom platforms has traditionally involved a fragmented manual workflow spanning specification interpretation, bytecode analysis, field mapping, and script-driven code generation. This thesis presents a technical study of the API Development Tool — a web-based platform developed during an internship at Amdocs — that consolidates this entire lifecycle into a single, guided interface. By pairing a React frontend with a Node.js/Express backend, the platform orchestrates Swagger/OpenAPI parsing, JAR bytecode introspection for EJB metadata extraction, interactive field-level mapping, YAML configuration generation, and automated module creation, delivering real-time progress feedback through Socket.IO event streaming. Through systematic source-code analysis and architectural evaluation, the study assesses the system across workflow coverage, parsing effectiveness, persistence correctness, security posture, and deployment readiness. The findings confirm strong capabilities in specification parsing, bytecode introspection, and end-to-end workflow orchestration, and further identify opportunities for enhancing the platform with authentication, cross-platform generation support, and automated testing infrastructure.

Contents

Acknowledgement	viii
Abstract	xi
1 Introduction	1
1.1 Motivation	2
1.2 Problem Context	4
1.3 Objectives	4
1.4 Method of Study	5
1.5 Report Organisation	5
2 Purpose and Scope	7
2.1 Purpose of the Project	7
2.2 Functional Scope	8
2.3 Persistence and Synchronisation	11

2.4	Non-Functional Considerations	12
2.5	Future Enhancement Opportunities	12
2.6	Practical Workflow Summary	13
3	Related Works	15
3.1	API-First and Contract-Driven Engineering	15
3.2	Existing Code Generation Ecosystems	16
3.2.1	Template-Based Generation Approaches	16
3.2.2	Model-Driven Code Generation	17
3.3	Bytecode Introspection and Metadata Extraction	17
3.4	Workflow Automation for Integration Teams	18
3.5	Real-Time Communication in Development Tools	19
3.6	Technology Stack Components	20
3.7	Comparison and Positioning	20
4	System Architecture and Design	22
4.1	Overview of the System	22
4.2	Frontend Architecture	23
4.3	Utility Layer: Swagger Parser	24
4.4	Utility Layer: JAR Parser and Bytecode Introspection	25
4.5	Utility Layer: Configuration Generator	27
4.6	Backend Architecture	30
4.7	Filesystem Synchronisation	31

4.8	Module Generation Pipeline	32
4.9	Deployment Architecture	32
4.10	Key Design Decisions	33
5	Results and Analysis	35
5.1	Introduction	35
5.2	Workflow Coverage Analysis	35
5.3	Architectural Assessment	36
5.4	Real-Time Communication Effectiveness	37
5.5	Persistence and Synchronisation Correctness	38
5.6	Design Strengths	39
5.7	Security Assessment	40
5.8	Deployment and Portability Assessment	40
5.9	Consolidated Findings	41
5.10	Evolution Readiness	43
5.11	Performance Observations	43
6	Conclusion and Future Work	45
6.1	Conclusion	45
6.2	Key Contributions	46
6.3	Future Work	47
6.3.1	Security Hardening	48
6.3.2	Cross-Platform Generation Support	48

6.3.3	Storage and Data Management Evolution	49
6.3.4	Testing Infrastructure	49
6.3.5	Frontend Architectural Improvement	49
6.3.6	Containerised Deployment	50
6.4	Scalability Considerations	50
6.5	Practical Applications	50
6.6	Summary	51
A	Biographical Sketch	53

CHAPTER 1

Introduction

Modern enterprise telecom platforms depend on a continually expanding catalogue of API connectors that bridge external digital services with internal business systems built around Enterprise JavaBean (EJB) architectures. Producing a new connector follows a broadly predictable lifecycle: an API specification in OpenAPI or Swagger format is ingested, its request and response schemas are analysed and mapped to the method signatures of enterprise service interfaces, configuration artifacts encoding those mappings are assembled, and code generation pipelines are invoked to produce a deployable module conforming to the organisation's Maven-based build infrastructure. Although each individual step is conceptually routine, the aggregate process is typically executed through a sequence of disparate manual activities — reading specifications in text editors, decompiling bytecode in Java development tools, recording mappings in spreadsheets, and invoking build scripts from the command line. As a result, connector implementations vary in quality, onboarding lead times remain elevated, and the surface area for human error grows with every new integration.

Against this backdrop, the present thesis offers a comprehensive technical study of an automated API connector development platform built to consolidate this fragmented lifecycle into a single, guided workflow. Developed during an internship at Amdocs, the platform pairs a React 18 frontend [5] with a Node.js/Express backend [4] to orchestrate the entire pipeline from specification upload to deployable module output. Through a structured web interface, users

upload Swagger or OpenAPI specification files, ingest compiled JAR artifacts containing enterprise service definitions, perform interactive field-level mapping between API schemas and EJB method parameters, generate YAML configuration files encoding those decisions, and trigger automated module creation through scripted build pipelines. Persistent state is managed through a hybrid of in-process SQLite storage and convention-based filesystem layout, while real-time progress feedback during generation reaches the browser through Socket.IO [8] event streaming.

What distinguishes this platform is not the novelty of any single component but rather the synthesis of specification parsing, bytecode introspection, interactive mapping, configuration assembly, and code generation into a coherent, tool-assisted workflow with full operational transparency. By unifying these activities within one interface, the system reduces context switching, enforces structural consistency across connector implementations, provides an auditable trail of mapping decisions, and transforms previously opaque batch operations into transparent, observable processes.

1.1 Motivation

Recurring operational challenges in the connector delivery pipeline of enterprise integration teams motivated the development of this platform. In the absence of dedicated tooling, each new connector demands manual interpretation of API specification documents that vary in structure, version, and complexity across providers. Engineers must then locate the correct EJB service interfaces within compiled Java client-kit archives, decipher method signatures and deeply nested parameter type hierarchies, and establish field-level correspondences between API payloads and enterprise service contracts — all without a centralised record of prior decisions.

Beyond being time-intensive, this manual mapping process is inherently error-prone. Different engineers may interpret the same specification differently, producing configuration outputs that diverge in naming conventions, field ordering, and structural layout. Because the YAML configuration files that drive downstream code generation are sensitive to structural errors, mistakes at the mapping stage propagate into generated source code, Maven build descriptors, and deployment artifacts. Compounding the problem, the generation process itself requires invoking platform-specific scripts with precise parameters and monitoring their output for failures, adding yet another layer of operational overhead to each delivery cycle.

Perhaps most critically, the absence of a centralised tool means that mapping decisions, configuration rationale, and generation history are never systematically captured. When connectors require updates or debugging, engineers must reconstruct the original context from scattered files and undocumented conventions, significantly increasing maintenance overhead. It was precisely this fragmented, ad-hoc reality that the platform studied in this thesis was conceived to replace with a structured, repeatable, and fully traceable workflow.

Table 1 summarises the key differences between the conventional manual approach and the tool-assisted workflow provided by the platform.

Table 1: Comparison of manual and tool-assisted connector development.

Aspect	Manual Workflow	Tool-Assisted Workflow
Specification analysis	Text editors, manual reading	Automated parsing with reference resolution and schema flattening
Service discovery	JAR decompilation in Java IDEs	Browser-based bytecode introspection without JVM
Field mapping	Spreadsheets, ad-hoc notes	Interactive dropdowns with source inference
Configuration authoring	Hand-written YAML files	Automated YAML generation from mapping state
Code generation	Command-line script invocation	One-click generation with real-time streaming
Traceability	Undocumented, scattered files	Full audit trail in database
Consistency	Varies by engineer	Enforced by structured workflow

1.2 Problem Context

Rather than targeting a greenfield microservice architecture, the platform operates within an established enterprise integration environment. This environment is characterised by a mature digital connector module tree, standardised Maven-based build infrastructure with custom code-generation plugins, enterprise client-kit JAR artifacts containing compiled EJB service definitions, and deployment constraints spanning Windows development workstations and Linux production servers that operate in air-gapped networks without internet access.

Such a setting imposes stringent constraints on any automation tooling. Generated connector code must conform to existing module directory structures, including prescribed locations for Swagger specifications under `src/main/resources/swagger/`, configuration files under `src/main/resources/configs/`, and Java source code aligned with Maven conventions. Generation scripts, in turn, must produce output compatible with the established build chain, including correctly parameterised Maven plugin configurations with per-API code-generation execution blocks. Above all, the platform must coexist with the surrounding toolchain without requiring modifications to the enterprise build infrastructure.

Consequently, the core technical challenge is not API parsing or code generation in isolation. It is *workflow continuity*: the ability to seamlessly connect specification ingestion, bytecode-driven metadata extraction, interactive field mapping, configuration generation, filesystem synchronisation, and module generation into a single, coherent operational control plane that respects every environmental constraint.

1.3 Objectives

At the broadest level, this thesis aims to conduct a thorough code-level and architecture-level study of the API connector development platform, examining its design decisions, implementation patterns, operational behaviour, and evolution potential. More specifically, the study seeks to document the complete data flow across the frontend, backend, filesystem, and generation layers, tracing how user inputs are transformed through each stage of the pipeline from specification upload to deployable module output.

In addition, the thesis evaluates the deployment model and runtime behaviour under realistic operational conditions — including the dual-server architecture with a Tomcat-hosted frontend

and a PM2-managed backend, the self-signed certificate lifecycle, and the filesystem-based storage strategy. Technical risks are identified and assessed across security, reliability, portability, and maintainability dimensions, yielding an evidence-based assessment with specific findings and severity classifications. Drawing on this analysis, the thesis proposes prioritised recommendations for production hardening and architectural evolution.

1.4 Method of Study

A repository-driven technical deep dive, conducted through systematic source code analysis and architectural review, forms the methodological foundation of this work. On the backend, the server implementation is examined at the route level, encompassing request handling for each endpoint group, database schema and migration logic, synchronisation routines, and generation orchestration. On the frontend, the analysis proceeds at the component and utility level, covering the `YamlUploader` workflow engine, the `ConfigViewer` editing interface, and the API communication layer.

Particular attention is devoted to the parser and generator utility modules. For the Swagger parser, the reference resolution algorithm with its `WeakMap`-based caching, the recursive schema flattening logic with circular reference detection, and the operation detail extraction mechanism are traced through their implementations. For the JAR parser, class file reading through `java-class-tools`, JVM descriptor and generic signature parsing, EJB interface detection through annotation and heuristic strategies, deep field path extraction through recursive type traversal, and synthetic parameter name normalisation are documented directly from the source code. Similarly, the configuration generator's path normalisation, source inference, response mapping aggregation, and multi-EJB call chain assembly are analysed for correctness and completeness.

Deployment documentation, path configuration modules, and the PowerShell build-and-package script are also reviewed to assess operational readiness. Every conclusion drawn in this thesis is evidence-backed and implementation-specific, grounded in observed source code behaviour rather than theoretical assumptions.

1.5 Report Organisation

The remainder of this report is organised as follows:

2. **Chapter 2: Purpose and Scope** — Defines the project intent, functional boundary, and expected outcomes. It documents the rationale behind the system design, describes the practical workflow, and clarifies the boundaries between in-scope functionality and identified future work.
3. **Chapter 3: Related Works** — Reviews existing approaches in API-first development, contract-driven code generation, bytecode introspection, and workflow automation for integration teams, positioning the present platform within this landscape and identifying the specific gap it addresses.
4. **Chapter 4: System Architecture and Design** — Presents a detailed walkthrough of the system architecture, covering the React frontend, Node.js/Express backend, utility layer parsers and generators, filesystem synchronisation strategy, module generation pipeline, and deployment model.
5. **Chapter 5: Results and Analysis** — Evaluates the system through functional and architectural analysis, examining workflow coverage, parsing effectiveness, real-time communication, persistence correctness, security posture, and deployment readiness, with a consolidated findings matrix summarising the assessment.
6. **Chapter 6: Conclusion and Future Work** — Summarises core contributions and proposes a phased roadmap for future enhancements spanning security features, cross-platform support, storage improvements, and testing infrastructure.

CHAPTER 2

Purpose and Scope

2.1 Purpose of the Project

Enterprise API connector delivery, as practised within large-scale telecom integration teams, has traditionally been a manual endeavour in which engineers independently interpret specifications, discover service interfaces, record field mappings, author configuration files, and invoke build scripts. Because each activity relies on individual expertise and produces artifacts in isolation, the resulting connectors tend to diverge in naming conventions, structural layout, and mapping accuracy, while the rationale behind design choices goes unrecorded. Over successive delivery cycles, this absence of systematic traceability makes maintenance increasingly difficult and raises the barrier for onboarding new team members.

The API Development Tool was conceived to address precisely this fragmentation. Its overarching purpose is to consolidate the full connector creation lifecycle — from specification upload through to deployable module output — within a single, guided web application. A React 18 frontend [5] presents a structured sequence of workflow steps: selecting or creating a connector module, uploading Swagger or OpenAPI specification files, ingesting enterprise client-kit JAR files for EJB metadata extraction, performing interactive field-level mapping between API payload schemas and EJB method parameters, generating YAML configuration files that encode those mapping decisions, and triggering automated module generation with real-time progress

streaming. On the server side, a Node.js/Express backend [4] orchestrates file storage, specification and bytecode parsing, database persistence, filesystem synchronisation, and generation script invocation.

By bringing these formerly isolated activities under a unified control plane, the platform materially reduces connector onboarding lead time, enforces structural consistency across generated configurations, and establishes a complete audit trail linking every specification upload, mapping decision, and configuration artifact to its provenance.

2.2 Functional Scope

The system's functional boundary spans five interconnected phases of the connector development lifecycle, each building upon the outputs of its predecessor. Figure 2.1 illustrates the sequential progression through these phases.

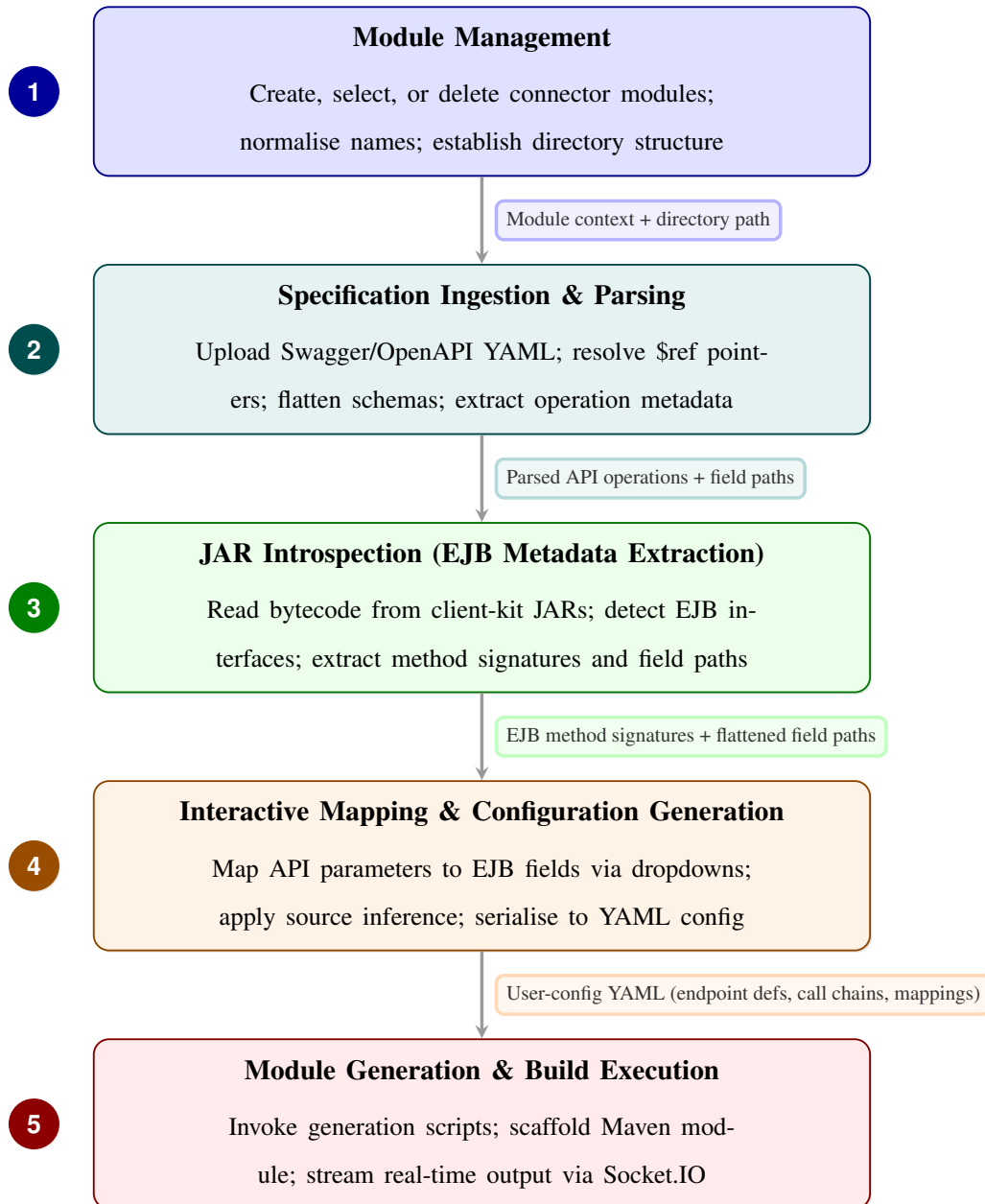


Figure 2.1: Five-phase workflow lifecycle of the API Development Tool, showing the key artifacts passed between each phase.

Module management constitutes the entry point of the workflow. Users create, list, select, or delete connector modules within the digital connector repository, where each module corresponds to a specific API integration and maintains its own directory structure for specifications, configurations, and generated source code. Module names are normalised to lowercase alphanumeric strings to prevent filesystem conflicts, and the directory tree itself serves as the authoritative source of truth for the module inventory — a design choice that eliminates synchronisation

ambiguity between database records and physical storage.

Once a module is established, the workflow advances to specification ingestion and parsing. The platform accepts API specification files in YAML format conforming to OpenAPI 2.0 (Swagger) or OpenAPI 3.0 standards. Upon upload, each specification is stored in the module's prescribed directory structure and registered in the metadata database. The frontend's Swagger parser utility then resolves JSON reference pointers using a `WeakMap`-based result cache, flattens nested schema definitions through recursive traversal with circular reference detection, and extracts structured metadata for every API operation — covering path parameters, header parameters, query parameters, request body fields, and response schemas with their associated model names.

With the API contract parsed, the next logical step involves client-kit JAR ingestion and EJB metadata extraction. Users upload enterprise JAR files containing compiled Java classes that define EJB service interfaces. The platform's JAR parser reads Java bytecode through the `java-class-tools` library, extracts class metadata by interpreting the constant pool and method descriptors, identifies EJB remote interfaces via annotation detection and naming-convention heuristics, and produces structured metadata encompassing method signatures, parameter types, return types, and deeply flattened field paths derived from recursive type-hierarchy traversal. Content-based deduplication using SHA-256 hashing ensures that identical JAR files are never stored redundantly.

Building on the metadata extracted in the preceding phases, the workflow proceeds to interactive field mapping and configuration generation. Users establish correspondences between API request parameters and EJB method inputs, and between EJB method return values and API response fields. The mapping interface presents dropdown selections populated from the parsed specification and bytecode metadata, enabling precise, auditable decisions. Notably, the platform supports multi-EJB call chains in which the response of one EJB invocation feeds into a subsequent call — a pattern common in enterprise integration scenarios where a single API endpoint aggregates data from multiple service methods. A source inference algorithm automatically suggests parameter mappings based on field name correspondence, accelerating the mapping process for specifications with predictable naming. Completed mappings are serialised into YAML configuration files conforming to the structure expected by the downstream generation pipeline.

The lifecycle culminates in module generation and build execution. Users trigger the generation pipeline through the interface, which invokes a PowerShell-based generation script on the

server. This script consumes the Swagger specification and user-config YAML to scaffold or update connector module source code, Maven build descriptors, and code-generation plugin configurations. Generation output is streamed to the frontend in real-time through Socket.IO events, providing immediate visibility into each stage of the process and enabling rapid diagnosis when errors arise.

2.3 Persistence and Synchronisation

Maintaining coherent state across browser sessions, server restarts, and external filesystem modifications demands a carefully considered persistence strategy. The platform addresses this requirement through a hybrid model that combines a `sql.js` in-process SQLite database [9] with filesystem-based artifact storage.

On the database side, three primary tables maintain the platform's metadata: `swagger_files` records specification file references, `client_kit_metadata` stores parsed EJB structures and JAR file hashes, and `user_configs` tracks configuration file references and associated mapping metadata. Unique indexes on the combination of file name and module name for specifications, and on file hashes for client kits, enforce data integrity and prevent duplicate entries at the storage layer.

Filesystem storage, meanwhile, follows a convention-based directory structure rooted at a configurable digital root path. Each connector module maintains subdirectories for Swagger specifications at `src/main/resources/swagger/` and configuration files at `src/main/resources/configs/`. Enterprise client-kit JAR files reside in a shared `client-kits/` directory accessible across all modules.

Ensuring that database records and filesystem artifacts remain synchronised requires active coordination through three complementary mechanisms. At startup, reconciliation routines scan the module directory tree, detect discrepancies between filesystem state and database records, insert missing entries, remove stale references, and auto-correct inconsistent file paths. During runtime, `chokidar` [3] filesystem watchers detect external changes to specification files, JAR files, and configuration YAML files, triggering database updates and Socket.IO event broadcasts to all connected clients. As a final safeguard, a thirty-second polling fallback on the frontend supplements the WebSocket channel to ensure eventual consistency in environments where persistent connections may be interrupted by firewalls or proxy servers.

2.4 Non-Functional Considerations

Several cross-cutting non-functional requirements shape the platform’s deployment model and operational characteristics.

From a transport security standpoint, the system supports both HTTPS and HTTP operation through an environment variable toggle. In the default HTTPS mode, the backend automatically generates self-signed TLS certificates on first startup using the `selfsigned` library, with the certificate common name derived from the `SSL_COMMON_NAME` environment variable. This approach enables encrypted transport without dependency on external certificate authorities — a pragmatic choice for air-gapped environments where obtaining externally signed certificates is infeasible. The frontend is packaged as a WAR file for deployment to an Apache Tomcat [2] application server, while the backend runs as a standalone Node.js process managed by PM2 for automatic restart and process supervision.

Portability and self-containment constitute equally important design priorities. The `sql.js` database runs entirely in-process without external dependencies; all npm packages are bundled within the deployment archive; and no runtime internet access is required. Path resolution logic in the backend’s storage configuration module automatically adapts to both Windows and Linux environments using relative path computation from the server root, enabling development on Windows workstations and seamless deployment to Linux production servers without code modifications.

The build and packaging process is itself automated through a comprehensive ten-step PowerShell script. This script validates toolchain dependencies, builds the React application with the backend API URL injected as an environment variable, packages the build output into a WAR file using Java’s `jar` utility, and bundles the WAR alongside the backend server code and all dependencies into a single deployable ZIP archive ready for distribution to target environments.

2.5 Future Enhancement Opportunities

The platform covers the core connector development lifecycle comprehensively and provides a solid foundation upon which additional enterprise capabilities can be built. User authentication and role-based access control represent a natural next step, extending the platform for use across broader organisational boundaries. Container-based deployment through Docker or orchestration via Kubernetes would further simplify provisioning and enable horizontal scaling.

Automated testing infrastructure — unit tests, integration tests, and end-to-end test suites — would complement the existing codebase as the platform matures. Multi-tenant isolation would enable multiple engineering teams to share a single deployment with appropriate data separation. These enhancements are well within the architectural capacity of the current design and represent logical extensions of the platform’s modular structure.

2.6 Practical Workflow Summary

To ground the preceding functional description in operational terms, this section traces a representative end-to-end usage scenario.

The workflow begins when a user opens the platform’s web interface and selects or creates a connector module from the module dropdown. Having established the working context, the user uploads a Swagger YAML specification file through the drag-and-drop upload interface. The backend stores the file in the module’s specification directory and registers it in the database, while the frontend simultaneously parses the specification to extract all available API operations with their request and response schemas.

Attention then turns to the enterprise service layer. The user uploads one or more client-kit JAR files, whereupon the backend computes SHA-256 hashes for deduplication, reads Java bytecode from each class file within the archive, identifies service interfaces and method signatures, resolves parameter and return type hierarchies through recursive traversal, and flattens field paths to produce comprehensive, navigable path lists. The parsed metadata is presented to the user, who selects the relevant EJB interfaces and methods for the target API operation.

With both the API schema and EJB metadata at hand, the user proceeds to the mapping phase. The interface presents source fields from the API specification alongside target parameters from the selected EJB methods, with dropdown menus populated from the available options. For unmapped parameters, the platform’s source inference algorithm suggests matches based on field name correspondence. For response mapping, the user establishes correspondences between EJB return value paths and API response schema fields, with support for multi-EJB response aggregation.

Upon completing the mapping, the user triggers configuration generation. The platform assembles a structured YAML configuration file encoding the module name, connector generation flags, endpoint definitions with EJB call sequences, per-EJB field mappings, and response mappings with model names and field correspondences. This configuration is saved to the module’s

configuration directory and registered in the database.

Finally, the user initiates module generation, which executes the PowerShell generation script with the Swagger and configuration files as inputs. The script operates in safe-add mode by default, appending new API endpoints to existing modules without overwriting previously generated code. Generation output streams to the frontend console in real-time through Socket.IO events, and the completion status is communicated through a terminal event containing the exit code and summary message.

CHAPTER 3

Related Works

3.1 API-First and Contract-Driven Engineering

Over the past decade, API-first development has matured from an aspirational design philosophy into a foundational engineering practice, particularly within enterprise integration environments where dozens of services must interoperate reliably across organisational and technological boundaries. Under this paradigm, the API specification functions as the primary design artifact from which all downstream implementation, testing, and documentation activities derive [7]. The OpenAPI Specification — formerly known as Swagger — has emerged as the de facto industry standard for describing RESTful APIs in a machine-readable format, defining a structured, language-agnostic representation of endpoints, request and response schemas, authentication requirements, and error models that enables automated tooling at every stage of the development lifecycle.

Contract-driven engineering extends this paradigm further by treating the API specification as a binding agreement between service producers and consumers. Changes to the specification are governed by versioning policies, and both client and server implementations are validated against the contract to ensure compatibility. This discipline proves especially critical in enterprise settings where APIs bridge legacy systems — such as EJB services deployed on application servers — with modern digital platforms. Any discrepancy between the API contract and

the underlying service implementation can propagate failures across dependent systems, making automated contract-to-code mapping not merely a convenience but an operational necessity.

The platform studied in this thesis operates squarely within this contract-driven paradigm. It consumes OpenAPI and Swagger specifications as input, extracts structured metadata through automated parsing with reference resolution and schema flattening, and channels this metadata into a configuration and code generation pipeline. Throughout the process, the specification serves as the single source of truth for API structure; every mapping decision and generation output traces back to the parsed contract.

3.2 Existing Code Generation Ecosystems

A substantial body of tooling already exists for generating API client and server code from specifications. The OpenAPI Generator project, for instance, provides a comprehensive framework for producing client libraries, server stubs, and API documentation in over forty programming languages from an OpenAPI definition [7]. Its predecessor, Swagger Codegen, pioneered template-based code generation using Mustache templating with support for customisable output structures. Both tools have achieved widespread adoption across the industry, significantly reducing the effort required to scaffold new API implementations and demonstrating that specification-driven automation can yield substantial productivity gains.

Yet these generic code generators are designed primarily for greenfield API development, where the generated code itself defines the implementation structure. In enterprise integration environments, the challenge is fundamentally different: generated code must conform to pre-existing module directory hierarchies, enterprise service conventions involving EJB interfaces and Maven plugin chains, and deployment-specific directory layouts that predate the API specification by years. Adapting generic generators to these constraints requires substantial post-generation customisation — often negating much of the automation benefit and reintroducing the very manual configuration steps the tools were meant to eliminate.

3.2.1 Template-Based Generation Approaches

Template-based code generation relies on parameterised templates that are populated with metadata extracted from API specifications. Swagger Codegen and OpenAPI Generator, for example, employ Mustache or Handlebars templates to produce language-specific source files. The primary advantage of this approach lies in its flexibility: templates can be customised to match

organisational coding standards, and new language targets can be introduced by creating additional template sets without modifying the core generation engine.

However, template-based systems typically generate flat file outputs and do not orchestrate multi-step workflows involving specification parsing, metadata extraction from compiled enterprise artifacts, interactive field mapping, configuration assembly, and build execution. Generation is treated as a standalone, stateless operation rather than as one stage within an integrated pipeline. The platform studied in this thesis addresses this limitation by embedding code generation within a broader workflow encompassing upstream specification analysis, bytecode introspection, and interactive mapping, with the generation step consuming artifacts produced by each preceding stage.

3.2.2 Model-Driven Code Generation

Model-driven engineering approaches — such as those rooted in the Eclipse Modeling Framework or domain-specific languages — adopt a more structured strategy for code generation. These systems define metamodels that capture a domain’s structure and semantics, then apply model transformations to produce implementation artifacts. When applicable, model-driven approaches offer strong guarantees about structural consistency and can automate complex multi-artifact generation from a single model definition.

For integration-focused use cases in which the primary input is already a well-defined specification format like OpenAPI, however, the overhead of metamodel design and tooling infrastructure may outweigh the benefits. Rather than introducing an additional abstraction layer, the platform studied here takes a pragmatic approach: it directly consumes OpenAPI specifications as its input model and applies domain-specific transformation logic to produce the configuration and code artifacts demanded by the enterprise build chain, dispensing with the need for a separate metamodel definition or specialised modelling tools.

3.3 Bytecode Introspection and Metadata Extraction

One of the platform’s most distinctive contributions is its use of Java bytecode introspection to extract EJB service metadata from compiled client-kit JAR files, entirely without access to the original source code. This capability draws on an established body of work in Java bytecode analysis, where compiled class files are parsed to extract class hierarchies, method signatures, field definitions, annotation metadata, and generic type signatures.

Libraries such as ASM, Javassist, and BCEL provide mature programmatic access to Java bytecode structures and are widely employed in dependency injection frameworks, aspect-oriented programming systems, and static analysis tools. These libraries, however, are Java-native and presuppose a JVM runtime. The platform takes a different path by adopting `java-class-tools`, a JavaScript-based class file reader that parses Java bytecode within the Node.js and browser runtime environments. This design decision enables both server-side and client-side JAR analysis without requiring a Java Virtual Machine installation — a particularly advantageous property in air-gapped deployment environments where Java tooling may not be available on every workstation.

The depth of the platform’s bytecode introspection merits particular attention, as it extends well beyond simple method signature extraction. The parser resolves class inheritance hierarchies to identify interface-to-implementation relationships and traces the superclass chain to assemble complete field inventories. It detects EJB remote interface annotations across both the `javax.ejb` and `jakarta.ejb` namespaces, falling back to heuristic naming pattern analysis when annotations are absent. It parses JVM generic type signatures to recover parameterised type information not available from raw method descriptors. Most critically, it extracts deeply nested field paths from parameter and return types through recursive type traversal with configurable depth bounds, producing comprehensive, navigable path hierarchies suitable for interactive mapping. As a final refinement, post-processing logic replaces compiler-generated synthetic parameter names with semantically meaningful alternatives derived from type information, enhancing the readability of the metadata presented to users.

3.4 Workflow Automation for Integration Teams

Beyond code generation, modern integration platforms increasingly incorporate workflow automation to orchestrate multi-step development processes. Continuous integration and deployment tools such as Jenkins, GitHub Actions, and Azure DevOps provide robust pipeline orchestration for build, test, and deployment automation. Their focus, however, lies predominantly on downstream stages of the development lifecycle; they do not typically address the upstream workflow of specification interpretation, metadata extraction, and configuration preparation that precedes the first line of generated code.

At the other end of the spectrum, low-code integration platforms such as MuleSoft Anypoint, Dell Boomi, and Microsoft Power Automate offer visual workflow designers for API integration. These platforms simplify connector development through drag-and-drop interfaces and

pre-built connectors, lowering the expertise threshold for basic integration tasks. Yet they operate at a substantially higher abstraction level and may not provide the fine-grained control needed when organisations maintain custom module structures, rely on legacy EJB service dependencies, and enforce organisation-specific Maven build chains.

The platform studied in this thesis occupies a deliberate middle ground between these two categories. It provides a guided workflow interface with interactive field mapping and dropdown-based parameter selection — conceptually akin to the usability principles of low-code platforms — while generating code and configurations compatible with the existing enterprise build and deployment infrastructure. This positioning addresses a genuine gap: full-featured integration platforms prove too heavyweight or abstract for teams with established, bespoke build conventions, while generic code generators are too narrow in their workflow coverage to replace manual processes end-to-end.

3.5 Real-Time Communication in Development Tools

As build and generation processes grow in complexity and duration, real-time communication capabilities in development tools have gained increasing importance. WebSocket-based communication, standardised in RFC 6455, enables persistent bidirectional connections between client and server, supporting use cases ranging from live log streaming and collaborative editing to real-time notification delivery.

Socket.IO [8], the library adopted by the platform, provides a high-level abstraction over WebSocket connections with automatic reconnection, event-based messaging, and transparent fallback to HTTP long-polling when WebSocket connections are unavailable. This reliability layer proves particularly valuable in enterprise network environments where firewalls, proxy servers, and security appliances may disrupt persistent WebSocket connections.

Within the platform, Socket.IO fulfils two complementary roles. During module generation, it streams script output to the frontend line-by-line, enabling users to observe the process in real-time and diagnose failures immediately rather than waiting for a batch completion report. In parallel, it broadcasts data synchronisation events whenever module, specification, or configuration state changes, ensuring that all connected clients maintain a consistent view of the system. This dual-purpose real-time channel replaces what would otherwise require periodic polling with immediate event-driven updates, while a thirty-second polling fallback on the frontend provides a safety net for environments where WebSocket reliability cannot be guaranteed.

3.6 Technology Stack Components

The platform’s implementation draws upon several established and widely adopted technology components, each selected for its suitability to the constraints of enterprise deployment.

React 18 [5] serves as the frontend framework, providing component-based UI development with efficient rendering through its virtual DOM reconciliation algorithm. The frontend employs React Router for client-side navigation between the primary upload workflow and the configuration viewer interface, with the application served under a `/retail` base path for compatibility with Tomcat WAR deployment.

On the server side, Express.js [4] provides the HTTP framework with middleware-based request processing, supporting JSON body parsing, CORS handling, and multipart file upload through Multer [6]. The sql.js library [9] delivers an in-process SQLite database compiled to WebAssembly, eliminating external database server dependencies while retaining relational data storage with full SQL query capabilities. Chokidar [3] provides cross-platform filesystem watching to detect changes to specification, JAR, and configuration files made outside the platform’s own interface.

The generation pipeline relies on Maven [1] for Java project compilation and PowerShell scripting for orchestrating the generation sequence, encompassing module scaffolding, Maven plugin configuration, and build invocation. The deployment stack combines Apache Tomcat [2] for serving the frontend WAR package with PM2 for Node.js process management and automatic crash recovery.

3.7 Comparison and Positioning

Drawing together the threads of the preceding discussion, several broad categories of tooling address aspects of the API integration workflow, each with distinct strengths and inherent limitations. Generic code generators such as OpenAPI Generator provide broad language coverage and template customisation but lack workflow orchestration, bytecode introspection, and enterprise-specific module structuring. Low-code integration platforms offer visual workflow design but may struggle to accommodate the custom generation pipelines required by environments with legacy module architectures and organisation-specific build chains. CI/CD tools deliver powerful build and deployment automation but do not address the upstream specification-to-configuration workflow.

The platform studied in this thesis synthesises specification parsing, bytecode-driven EJB meta-data extraction, interactive field mapping, YAML configuration generation, and scripted module creation into a single guided workflow with real-time observability. Unlike generic generators, it produces output conforming to existing enterprise module structures and treats compiled artifacts as first-class inputs. Unlike low-code platforms, it operates directly atop the organisation's established build infrastructure. This combination of guided workflow, enterprise-specific generation, bytecode introspection, and real-time transparency carves out a distinctive niche among existing approaches, with a modular architecture that readily accommodates future enhancements such as authentication, containerised deployment, and automated testing.

Table 2 consolidates this comparative analysis across the key capability dimensions.

Table 2: Feature comparison of existing approaches and the API Development Tool.

Capability	OpenAPI Gen.	Low-Code	CI/CD	This Platform
Spec parsing	✓	✓	–	✓
Bytecode intro- spection	–	–	–	✓
Interactive map- ping	–	Partial	–	✓
Config generation	–	Partial	–	✓
Code generation	✓	✓	–	✓
Workflow orches- tration	–	✓	✓	✓
Real-time feed- back	–	Partial	✓	✓
Enterprise mod- ule conformance	–	–	–	✓
Air-gapped de- ployment	✓	–	Partial	✓

CHAPTER 4

System Architecture and Design

4.1 Overview of the System

Architecturally, the API Development Tool adopts a two-tier client-server model composed of three functional layers: a React frontend serving as the workflow engine, a Node.js/Express backend acting as the orchestration and persistence layer, and a digital connector repository providing the enterprise module tree, generation scripts, and Maven build infrastructure. Communication between the frontend and backend follows two complementary patterns — request-driven REST interactions over HTTPS for CRUD operations, and stream-driven WebSocket messaging via Socket.IO for real-time event delivery during long-running generation processes. The backend, in turn, interacts with the filesystem for artifact storage and with the digital connector repository for module generation, bridging the platform’s web-based interface with the enterprise build chain.

This layered separation is not incidental; it reflects a deliberate design choice to keep the user-facing workflow logic isolated from the server-side orchestration concerns, allowing each layer to evolve independently while maintaining well-defined communication contracts. [Figure 4.1](#) presents a high-level view of this architecture.

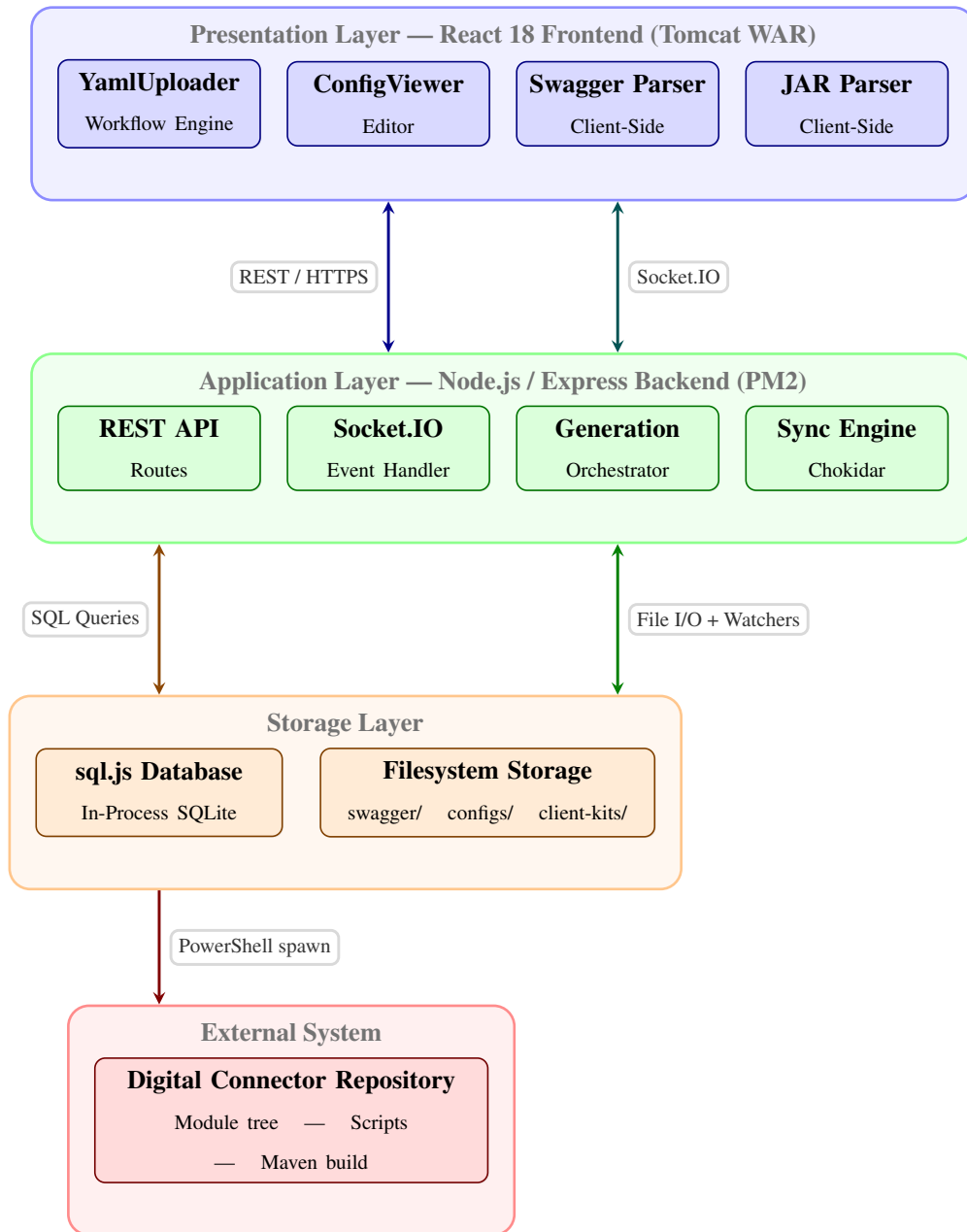


Figure 4.1: Layered system architecture of the API Development Tool, showing internal subcomponents, communication protocols, and deployment boundaries.

4.2 Frontend Architecture

The frontend is implemented as a single-page application using React 18 [5] with React Router for client-side navigation. The application shell, defined in `App.jsx`, establishes a `BrowserRouter` with the `/retail` base path and defines two primary routes: the root

route renders the `YamlUploader` component, which serves as the primary workflow engine, while the `/configs` route renders the `ConfigViewer` component for browsing and editing previously generated configurations.

Central to the frontend’s design is the `YamlUploader` component, which functions as the principal orchestrator of the connector development workflow. It manages substantial state spanning the selected module, uploaded Swagger files, parsed API operation metadata, uploaded client-kit JARs, extracted EJB structures, field mapping selections, response mapping state, generation status, `Socket.IO` connection state, and real-time console output. Rather than distributing this state across multiple loosely coupled components, the design collocates all workflow concerns within a single component, yielding a cohesive user experience where the entire connector creation process is visible and manageable from one place. On mount, the component initialises a `Socket.IO` connection, subscribes to synchronisation events for modules, specifications, client kits, and configurations, and establishes a thirty-second polling fallback to ensure eventual consistency even when `WebSocket` connections are disrupted by firewalls or proxy servers.

The frontend’s API communication layer is centralised in a dedicated service module that wraps all backend interactions through a unified fetch-based request function. To accommodate different deployment scenarios, the API base URL is resolved through a configuration module supporting three resolution strategies in priority order: an environment variable override baked into the build at compile time, a development proxy passthrough using an empty string, and runtime auto-detection that constructs the URL from the browser’s hostname with port 4000.

Complementing the primary workflow, the `ConfigViewer` component provides a secondary interface for reviewing and modifying previously generated configurations. It loads all saved configurations from the backend, groups them by module, and presents an editable view where users can adjust field mappings and regenerate the YAML output. Importantly, it reuses the same Swagger parser and configuration generator utilities as the primary workflow, guaranteeing consistency between the initial configuration creation and any subsequent editing.

4.3 Utility Layer: Swagger Parser

The Swagger parser utility is responsible for extracting structured metadata from parsed OpenAPI and Swagger specification documents. Its logic is organised around three interdependent algorithms — reference resolution, recursive schema flattening, and operation detail extraction

— each building upon the output of its predecessor.

Reference resolution handles the `$ref` mechanism that OpenAPI specifications employ pervasively to avoid schema duplication. When a property or parameter references another definition through a JSON reference pointer such as `#/definitions/User` or `#/components/schemas/User`, the parser splits the reference string into path segments, traverses the document tree, and retrieves the referenced schema object. To prevent redundant traversals, results are cached using a `WeakMap` keyed on the parsed document object — a choice that ensures repeated references to the same definition are resolved only once per document, while allowing the cache to be automatically garbage-collected once the document falls out of scope.

Recursive schema flattening builds on reference resolution to traverse a schema definition and produce a flat list of all property paths in dot notation, such as `address.street` or `orders[].id`. The algorithm processes direct object properties, follows `$ref` pointers to resolve referenced schemas, merges fields from composition operators (`allOf`, `oneOf`, `anyOf`), recurses into nested object types, and handles array types by appending bracket notation before recursing into item schemas. Circular references are detected through a visited-set that tracks previously processed schemas, and a depth limit of twelve levels provides an additional safety bound against unbounded recursion. The output is deduplicated at the root level to yield a clean set of unique field paths.

Operation detail extraction combines the preceding two algorithms to produce a comprehensive description of each API endpoint. For a given path and HTTP method, the parser extracts path parameters, header parameters, query parameters, form data parameters, request body fields, and response schemas with their model names. Parameters defined at both the path level and the operation level are merged using a composite key formed from the parameter's location and name, with operation-level definitions taking precedence over path-level ones. The parser handles both Swagger 2.0 style body parameters with inline schemas and OpenAPI 3.0 style request bodies with content-type-keyed schemas, adapting its extraction logic to the specification version automatically.

4.4 Utility Layer: JAR Parser and Bytecode Introspection

The JAR parser implements a bytecode introspection pipeline that extracts EJB service metadata from compiled Java Archive files without requiring access to the original source code or a Java Virtual Machine installation. Both a browser-side variant and a server-side variant exist, sharing

the same algorithmic core but adapted for their respective runtime environments.

Parsing begins by reading the JAR file as a ZIP archive through the `jszip` library. Each `.class` file within the archive is extracted in batches of twenty-four for concurrent processing and fed to the `java-class-tools` class file reader. This reader processes the Java class file binary format to extract the constant pool, class access flags, field definitions, method definitions, and attribute tables — the raw material from which all higher-level metadata is derived.

For each class file, the parser resolves the fully qualified class name, superclass name, and implemented interfaces from constant pool references. Field descriptors are parsed using the JVM descriptor grammar, a compact encoding where `L` prefixes object types, `[]` denotes array dimensions, and single characters represent primitive types (`I` for integer, `Z` for boolean, and so forth). Method descriptors undergo analogous parsing to extract parameter type sequences and return types. When generic type signatures appear in the class file's `Signature` attribute, the parser recovers parameterised type information that raw descriptors cannot express — enabling the system to determine, for example, that a field declared as `java.util.List` actually contains elements of a specific domain type.

EJB service interface identification employs a two-tier strategy designed to maximise detection recall without sacrificing precision. The primary mechanism inspects the `RuntimeVisibleAnnotations` attribute for `javax.ejb.Remote` or `jakarta.ejb.Remote` annotations. When annotations are absent — a common scenario with older or partially annotated codebases — the parser falls back to naming pattern heuristics, checking interface names for suffixes such as `Svc`, `Remote`, `Facade`, and `Manager` that conventionally indicate service interface semantics, while filtering out common non-service interfaces like `java.io.Serializable`.

Among the parser's most distinctive capabilities is deep field path extraction. For each method parameter and return type, it recursively traverses the type hierarchy by resolving fields from the class definition, following superclass chains to incorporate inherited fields, deriving virtual fields from getter methods when a class declares few fields directly, and resolving collection element types from generic signatures. To prevent combinatorial explosion in deeply nested enterprise data models, this traversal is bounded by configurable depth limits — three levels for parameter expansion and eight for return type expansion — with an additional per-flatten cap of three hundred paths. The resulting field paths enable the mapping interface to present specific, navigable paths into complex enterprise data structures rather than opaque top-level type names.

As a final refinement, post-processing logic normalises synthetic parameter names generated by the Java compiler. Names such as `arg0`, `parameter1`, or `p0` are replaced with semantically meaningful alternatives derived from the parameter’s type information — for example, a parameter of type `SearchCustomerFilterInfoInput` with a compiler-generated name would be renamed to `searchCustomerFilterInfoInput`. Additional heuristics consult companion “Inputs” classes to recover meaningful parameter names when the primary class file lacks debug information.

4.5 Utility Layer: Configuration Generator

The configuration generator bridges the gap between interactive user mapping decisions and the structured YAML format expected by the downstream generation pipeline. Its responsibilities span several mapping concerns: path parameter extraction from API path patterns, EJB method parameter mapping with path normalisation, response field mapping with interface-to-implementation resolution, and multi-EJB call chain assembly.

On the request mapping side, the generator processes each mapping entry to resolve source paths from the API specification and target paths into EJB method parameters. Path normalisation strips Java member prefixes such as `m_`, converts collection type suffixes to array notation, and enforces camelCase conventions. When users leave a parameter unmapped by selecting “none”, the generator’s source inference algorithm searches available sources for field name matches, preferring nested paths over flat parameter names when multiple candidates exist. Path rewriting logic handles the common scenario where an EJB interface name differs from its implementation class name, replacing interface-derived prefixes with implementation-derived prefixes throughout the mapping to ensure compatibility with the generated code.

For response mapping, the generator resolves EJB return type paths, filters out JVM internal fields and serialisation artifacts such as `serialVersionUID` and framework-specific dirty flags, and derives canonical address-style paths from deeply nested return structures. It also produces relative element paths from array return types, making deeply nested response fields accessible through shorter, more intuitive path expressions. The response mapping logic handles multi-EJB scenarios where the response of one call feeds into a subsequent call, with prior EJB response paths made available as mapping sources for later inputs.

The final output is a structured YAML document comprising the module name, connector generation flags, endpoint definitions with EJB call sequences, per-EJB field mappings, and response

mappings with model names and field correspondences. Serialisation is performed through the `js-yaml` library with settings optimised for readability — two-space indentation and a generous line width.

Figure 4.2 summarises the end-to-end data flow through these three utility modules, illustrating how raw inputs are progressively transformed into the configuration artifacts that drive module generation.

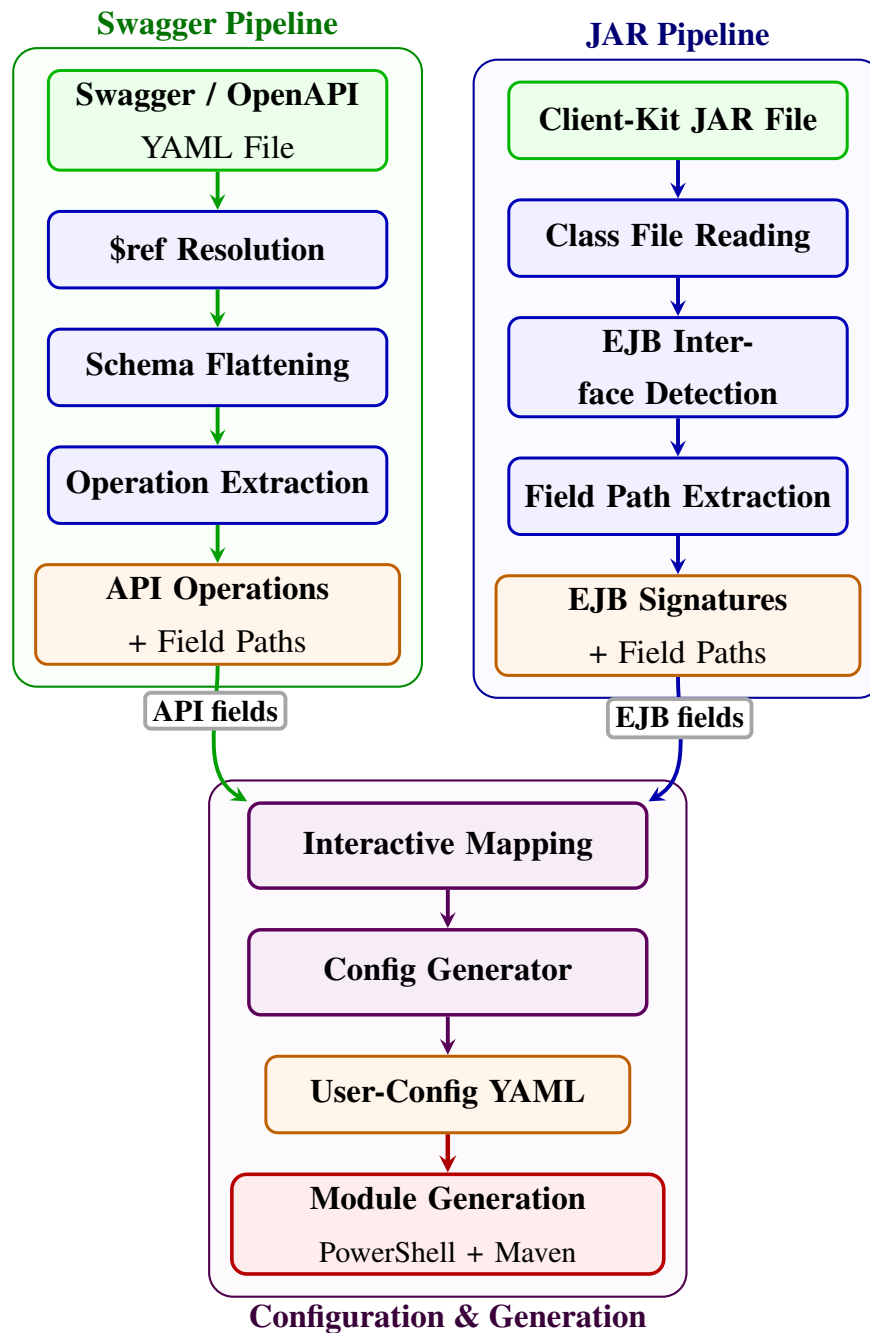


Figure 4.2: Data flow through the utility layer: two parallel parser pipelines (Swagger and JAR) feed extracted field information into the interactive mapping stage, which produces the user-config YAML that drives final module generation.

4.6 Backend Architecture

The backend is built on Express.js [4] and organised as a monolithic server module that unifies REST API routing, database management, file storage, real-time communication, and external process orchestration. Its startup sequence proceeds through a well-defined initialisation chain: loading environment configuration through `dotenv`, computing all storage paths from the configurable digital root, creating required directories when absent, initialising the `sql.js` database with schema creation and migration logic (including table creation and index establishment), running startup reconciliation routines for module files, client kits, and user configurations, and finally starting the HTTP or HTTPS server with an attached `Socket.IO` instance.

The REST API surface is organised into five functional groups: module management endpoints for creating, listing, and deleting connector modules; Swagger endpoints for uploading, retrieving, listing, and deleting specification files; client-kit endpoints for uploading, listing, enriching, and deleting JAR files together with their parsed metadata; configuration endpoints for generating, saving, listing, retrieving, and updating user-config YAML files; and generation endpoints for triggering and monitoring module generation. A dedicated health check endpoint provides service status verification with path diagnostics.

Underpinning the API surface, the database layer employs `sql.js` [9] — a pure JavaScript implementation of SQLite compiled to WebAssembly — running entirely within the Node.js process. This architectural choice eliminates external database server dependencies, a critical consideration for air-gapped deployment. Three primary tables — `swagger_files`, `client_kit_metadata`, and `user_configs` — store the platform’s relational metadata, with unique indexes preventing duplicate entries and foreign key references linking configurations to their source artifacts. Each database mutation triggers a synchronous export of the in-memory database to a filesystem-backed file through the `saveDb` function, ensuring reliable persistence across server restarts with crash-safe durability.

Input validation is implemented at multiple levels to mitigate injection and path traversal risks. File names are validated against allowed extensions and checked for path traversal attempts using a `resolvePathInside` utility that verifies resolved paths remain within expected base directories. Module names are normalised to lowercase alphanumeric strings with underscores and hyphens. JAR file names are validated for length, allowed characters, and format.

4.7 Filesystem Synchronisation

Maintaining consistency between database records, local filesystem artifacts, and the digital connector repository constitutes one of the platform's more demanding operational requirements. The backend addresses this through a multi-layered synchronisation strategy that operates at two temporal scales: startup-time reconciliation and runtime-continuous monitoring.

At startup, three reconciliation routines execute in sequence. The `syncModuleFiles` routine iterates over each module, reads the specification directory contents, compares them against database records, inserts entries for files present on disk but absent from the database, and removes database entries whose corresponding files no longer exist — cascading the deletion to any associated user configurations. The `syncClientKits` routine performs analogous reconciliation for JAR files, including cross-referencing hash-based deduplication records. The `syncUserConfigs` routine scans configuration directories, imports externally created configuration files, and purges entries pointing to non-existent files.

During runtime, `chokidar` [3] filesystem watchers monitor three directories with tailored configurations: the module directory tree at six levels of depth for specification file changes, the client-kits directory for JAR additions and removals, and the configuration directory for YAML file changes. Raw file change events are debounced with stabilisation thresholds of four to five hundred milliseconds to prevent duplicate processing during file writes. When changes are confirmed, the watchers trigger database updates and broadcast `Socket.IO` synchronisation events to all connected clients. At the module level, a pending-syncs map with five-hundred-millisecond timeouts coalesces rapid successive changes into a single synchronisation pass.

Connected clients subscribe to `Socket.IO` events following a domain-based naming convention. Synchronisation events include `modules:sync`, `swaggers:sync`, `clientkit:sync`, and `configs:sync`. Generation events include `generate:start`, `generate:output`, and `generate:done`. On initial connection, the server immediately emits the current module list, client-kit inventory, and configuration list to bring the new client to a consistent state. Clients may also request targeted synchronisation through events such as `request:swaggers` and `request:configs`.

4.8 Module Generation Pipeline

The module generation pipeline bridges the platform’s configuration output with the enterprise build chain, translating the mapping decisions captured in YAML configuration files into deployable connector module code.

When a user triggers generation, the backend resolves the paths to the Swagger specification and user-config YAML file, discovers the generation script in the digital connector repository through a glob-based search for files matching the `create-new-api-module*.ps1` pattern, and spawns a PowerShell child process with the appropriate parameters using Node.js’s `spawn` function. Two operational modes are supported: `safe-add` mode (the default in the platform’s workflow) appends new API endpoint configurations to an existing module without modifying previously generated code, while `force` mode rebuilds the entire module from scratch, removing existing generated artifacts before regenerating. A `skip-build` flag can bypass the Maven compilation step when only source file generation is required.

During execution, the script’s standard output and standard error streams are captured line-by-line through buffered stream processing. Each complete line is forwarded to connected clients through `generate:output` Socket.IO events, enabling the frontend to render a scrolling console that mirrors the generation process in real-time. Upon completion, the backend analyses the output for success markers such as “BUILD SUCCESS” and failure indicators such as “BUILD FAILED”, then emits a `generate:done` event carrying the final status, a summary message, and the module name. A ten-minute timeout on the spawn operation provides a safety bound against hung processes.

On non-Windows platforms where PowerShell is unavailable, the generation endpoint returns a graceful HTTP 501 response explaining that generation requires PowerShell, while confirming that all other platform features remain fully functional — a pragmatic degradation strategy appropriate for heterogeneous development environments.

4.9 Deployment Architecture

The deployment architecture follows a two-service model optimised for air-gapped enterprise environments. The React frontend is packaged as a WAR file deployed to Apache Tomcat [2], while the Node.js backend runs as a standalone HTTPS service managed by PM2. The frontend WAR contains the optimised React build output alongside a `WEB-INF/web.xml` descriptor

and a `META-INF/MANIFEST.MF` file.

Automating the build and packaging process is a comprehensive ten-step PowerShell script that validates toolchain dependencies (Node.js, npm, Java JDK, and PowerShell), verifies the project directory structure, optionally cleans previous build artifacts, installs frontend npm dependencies if absent, configures the backend API URL as an environment variable, runs the React production build through craco, verifies the backend URL's presence in compiled JavaScript bundles, packages the build output into a WAR file using Java's `jar` command, optionally executes API smoke tests against the backend, and bundles the WAR together with the backend server code and all dependencies into a single deployable ZIP archive. The script accommodates configurable environment profiles, custom backend URL injection, auto-detection mode for runtime URL resolution, and optional post-build smoke testing.

Path resolution is handled by a dedicated storage paths module that derives all filesystem paths from a configurable digital root directory. The module resolves the digital root as a sibling directory to the server by default and computes all downstream paths — the modules directory, per-module specification and configuration directories, the client-kits directory, and the metadata database path — from this single root. This approach enables the same codebase to operate correctly in both Windows development and Linux production environments without hardcoded path dependencies.

4.10 Key Design Decisions

Several deliberate design decisions shape the current implementation, each reflecting a conscious prioritisation of simplicity, deployment ease, and operational self-sufficiency — qualities that are essential for air-gapped enterprise environments.

Filesystem-based artifact storage provides simplicity, direct auditability through standard file system tools, and seamless compatibility with existing version control workflows. Engineers can inspect, back up, and version-control connector artifacts using familiar tooling without requiring specialised database administration. The synchronous database write pattern, where each mutation triggers a full in-memory database export through `sql.js`, ensures strong durability guarantees and crash-safe persistence, well suited to the platform's single-team usage model.

Concentrating workflow logic within the `YamlUploader` component yields a cohesive user experience in which all workflow state is collocated, transitions are immediately visible, and the entire connector creation process is managed through a single, unified interface. The choice of

PowerShell for generation script orchestration reflects compatibility with the existing enterprise toolchain and the Windows-based development environment where connectors are authored, while the backend includes graceful handling for non-Windows platforms.

Most significantly, the elimination of any external database dependency means the platform can be deployed to air-gapped environments without database server infrastructure, network access, or external service dependencies. Combined with bundled npm packages, in-process SQLite, and auto-generated self-signed certificates, this design achieves a fully self-contained deployment model that requires no runtime prerequisites beyond Node.js and a Java runtime.

CHAPTER 5

Results and Analysis

5.1 Introduction

This chapter presents the results of a comprehensive technical evaluation of the API Development Tool, drawing on systematic source code analysis, architectural review, and an assessment of operational behaviour under representative conditions. The evaluation spans workflow completeness, architectural quality, parsing and introspection effectiveness, real-time communication reliability, persistence and synchronisation correctness, security posture, deployment readiness, and evolution readiness. Each finding is grounded in evidence from the codebase, yielding a thorough picture of a well-engineered platform with strong capabilities across its target use cases.

5.2 Workflow Coverage Analysis

A thorough examination of the backend route structure and frontend workflow logic confirms that the platform implements a complete end-to-end connector development pipeline, spanning all five phases: module management, specification ingestion, client-kit introspection, interactive mapping with configuration generation, and automated module generation. Each workflow phase is backed by dedicated REST API endpoints, database operations, and frontend state management logic, ensuring that no stage of the connector lifecycle falls outside the tool's

reach.

The specification parsing pipeline handles both OpenAPI 2.0 (Swagger) and OpenAPI 3.0 format variations without user intervention. Its reference resolution algorithm correctly follows `$ref` pointers across nested definition hierarchies, with the `WeakMap`-based cache guaranteeing that repeated references within a single document are resolved in constant time after the initial traversal. Schema flattening produces comprehensive field path lists encompassing properties from composed schemas via `allOf`, `oneOf`, and `anyOf` operators, while circular reference detection prevents infinite recursion. Parameter merging correctly enforces the precedence rule whereby operation-level parameters override path-level parameters sharing the same location and name.

Equally noteworthy is the JAR bytecode introspection pipeline, which demonstrates strong capability in extracting actionable metadata from compiled enterprise artifacts. The parser correctly resolves JVM method descriptors and generic type signatures, identifies EJB remote interfaces through both annotation detection across the `javax.ejb` and `jakarta.ejb` namespaces and naming convention heuristics, and produces deeply flattened field paths from complex type hierarchies with configurable depth bounds. The post-processing step that replaces synthetic parameter names with type-derived alternatives — supplemented by companion class consultation for parameter name recovery — significantly improves the usability of the mapping interface by presenting human-readable names rather than compiler-generated placeholders.

Rounding out the pipeline, the configuration generator produces well-structured YAML output that correctly encodes multi-EJB call chains, field-level request and response mappings, method parameter metadata, and interface-to-implementation prefix rewriting. Its source inference logic for unmapped parameters reduces the manual mapping burden by automatically detecting field name correspondences across available options, a convenience that proves especially valuable for specifications with predictable naming conventions.

5.3 Architectural Assessment

The two-tier architecture provides a clear separation between the frontend presentation and workflow layer on one side and the backend orchestration and persistence layer on the other. On the backend, the REST endpoint organisation follows domain-driven grouping, with module, Swagger, client-kit, configuration, and generation operations encapsulated in logically distinct route groups. This organisation supports maintainability by enabling independent reasoning

about each functional domain and confining the impact of changes to their respective route groups.

On the frontend, centralising workflow logic in the `YamlUploader` component yields a cohesive user experience in which all state is collocated and transitions are managed through a single component tree. This design provides comprehensive workflow coverage within a unified orchestrator, managing module selection, file upload state, parsed specification data, JAR metadata, EJB selections, mapping entries, response mapping state, generation status, `Socket.IO` connection state, console output buffer, and polling timer state. The architecture is well structured for progressive decomposition into smaller, composable subcomponents as the platform evolves, with each concern cleanly separable into custom React hooks.

The utility layer exhibits strong separation of concerns. The Swagger parser, JAR parser, and configuration generator each encapsulate distinct algorithmic responsibilities and communicate through well-defined data structures. Crucially, the parser utilities are architecturally shared between browser and server environments through environment-specific adapters — the browser variant consuming file input buffers, the server variant accepting Node.js Buffer objects. This sharing eliminates divergence between client-side preview parsing and server-side persistence parsing, an important property for ensuring consistent behaviour across the platform.

The `Socket.IO` integration is layered alongside the REST API, utilising the same HTTP server instance while maintaining separate event-based communication channels. This dual-channel model — REST for request-response operations and `WebSocket` for event streaming — appropriately matches the two distinct interaction patterns present in the workflow.

5.4 Real-Time Communication Effectiveness

The real-time communication implementation demonstrates thoughtful design well suited to its primary use cases. During module generation, each line of script output is forwarded as an individual `Socket.IO` event, enabling the frontend to render a responsive, scrolling console that mirrors the generation process without the latency penalties inherent in polling-based approaches.

Beyond generation, the synchronisation event system for module, Swagger, client-kit, and configuration state changes provides immediate UI updates whenever artifacts are modified — whether through the backend API or through external filesystem operations detected by

chokidar watchers. The thirty-second polling fallback ensures eventual consistency in environments where WebSocket connections are disrupted by network infrastructure, while debounced watcher event processing, with stabilisation thresholds of four to five hundred milliseconds and module-level coalescing through pending-sync timeouts, prevents event flooding during rapid successive file operations.

Socket.IO's automatic reconnection mechanism provides resilience against transient connection failures. The initial connection state synchronisation — whereby the server emits current module lists, client-kit inventories, and configuration summaries to every newly connected client — ensures that clients joining mid-session receive a complete state snapshot. Taken together, this hybrid socket-plus-polling approach represents a pragmatic reliability strategy appropriate for enterprise network environments with variable connectivity characteristics.

5.5 Persistence and Synchronisation Correctness

The `sql.js` persistence layer correctly implements schema creation, migration, and data integrity enforcement through unique indexes and constraint validation. The three-table schema provides appropriate relational structure for the platform's data model: the unique index on the combination of file name and module name for the `swagger_files` table prevents duplicate specification uploads within the same module, while content-based deduplication of client-kit JARs through SHA-256 hashing — enforced by a unique index on the `file_hashes` column — prevents both storage waste and metadata inconsistency.

Particularly commendable are the startup reconciliation routines, which demonstrate defensive programming practices. Upon initialisation, the server systematically scans the module directory tree, identifies files present on disk but absent from the database, inserts missing records, removes database entries whose corresponding files no longer exist, cascades deletions to dependent user configurations, and auto-corrects stale file paths where a file has been relocated but remains discoverable. This self-healing behaviour ensures that the database converges to a consistent state after manual file operations, version control updates, or crash recovery — a robustness property that proves invaluable in an environment where engineers routinely interact with the filesystem directly.

The synchronous database write pattern, where each mutation triggers a full database export to disk through `sql.js`'s `export()` method, provides strong durability guarantees with crash-safe persistence. Every state change is immediately committed to disk, ensuring that no data

is lost even in the event of an unexpected server termination. This approach delivers reliable, predictable behaviour well suited to the platform's team-scale usage pattern.

5.6 Design Strengths

Several design decisions merit recognition for their positive contribution to the platform's overall effectiveness.

Foremost among these is the unified workflow approach, which consolidates what would otherwise be a fragmented series of manual activities — specification analysis in text editors, JAR decompilation in Java tools, mapping in spreadsheets, and command-line script invocation — into a single guided process. This consolidation reduces context switching, minimises missed steps, and ensures that all mapping decisions are captured in a machine-readable configuration format amenable to version control and auditing.

The real-time generation feedback mechanism transforms a previously opaque batch process into a transparent operation. Users observe each generation step as it executes, identify errors at the point of occurrence, and gain insight into the generation sequence without accessing server-side logs. This visibility improves confidence in the generated output and markedly accelerates failure diagnosis.

Equally significant is the bytecode introspection capability, which eliminates the need to manually examine Java source code or decompiled class files to understand EJB service interfaces. By automatically extracting method signatures, parameter types, and deeply nested field paths from compiled artifacts, the platform reduces the expertise barrier for connector development and enables engineers unfamiliar with the target services to create accurate mappings.

The air-gapped deployment support deserves particular mention: the platform operates without any external network dependencies whatsoever. The `sql.js` database runs entirely in-process, all npm dependencies are bundled in the deployment archive, self-signed certificates are generated on first startup, and no runtime internet access is required. This self-containment enables deployment to security-restricted environments where network connectivity is either unavailable or strictly prohibited.

5.7 Security Assessment

The platform's security posture is aligned with its design intent as an internal development tool operating within a trusted network boundary. The backend REST API provides open access to all endpoints, enabling frictionless usage by the engineering team without the overhead of credential management or session handling — an appropriate design for controlled internal environments.

File upload handling incorporates validation for allowed extensions and filename sanitisation through the `resolvePathInside` utility, which verifies that resolved paths remain within expected base directories, providing protection against path traversal attacks. The generation endpoint validates its input parameters before invoking the generation script, ensuring that only well-formed requests reach the underlying build pipeline.

Transport security is addressed through automatic self-signed TLS certificate generation on first startup, enabling encrypted HTTPS communication without dependency on external certificate authorities. The platform's architecture is well prepared for the addition of authentication middleware (such as JWT-based token verification) and fine-grained CORS policies as future enhancements, should the deployment scope extend beyond the current trusted network boundary.

5.8 Deployment and Portability Assessment

The deployment model has been validated through successful operation on a Linux production server in an air-gapped environment. The path resolution module correctly adapts filesystem paths between Windows development and Linux production environments using relative path computation from the server root and platform-agnostic path joining. PM2 provides automatic restart on crash, and self-signed certificate generation enables HTTPS operation without external certificate infrastructure.

The generation pipeline leverages PowerShell scripting, reflecting compatibility with the Windows-based enterprise toolchain. On Linux deployments, the generation endpoint provides a graceful response indicating the platform environment, while all other features — specification upload, JAR parsing, configuration generation, and data management — continue to function normally. This thoughtful separation of concerns ensures that the platform remains useful across heterogeneous environments, with generation capability available on the primary development platform.

The automated build-and-package script streamlines deployment preparation, producing a self-contained bundle that includes the frontend WAR, backend code, all npm dependencies, and Linux environment configuration templates. Its validation steps — toolchain checks, structure verification, and optional smoke testing — reduce the risk of deploying incomplete or misconfigured artifacts.

5.9 Consolidated Findings

The following table summarises the evaluation findings across the key dimensions assessed in this analysis.

Table 3: Consolidated findings matrix across evaluation dimensions.

Dimension	Assessment	Key Observation
Workflow Coverage	Strong	Complete end-to-end pipeline from specification upload through module generation
Architecture	Good	Clean separation between frontend workflow, backend orchestration, and utility layer
Parsing & Introspection	Strong	Comprehensive Swagger resolution and deep bytecode field path extraction
Real-time Feedback	Strong	Socket.IO streaming with debounced watchers and polling fallback
Persistence	Good	sql.js with startup reconciliation, hash-based deduplication, and self-healing sync
Security	Appropriate	Trusted-network design with path validation and HTTPS encryption
Portability	Good	Cross-platform path resolution with graceful environment adaptation
Testing	Extensible	Architecture supports progressive addition of test suites
Scalability	Fit for Purpose	Self-contained single-server model optimised for team-scale deployment

5.10 Evolution Readiness

A notable strength of the platform’s architecture is its readiness for progressive enhancement. The modular backend route structure allows new functional domains to be added as independent route modules without disrupting existing functionality. The frontend’s component architecture supports additional workflow steps through independent component development. The well-defined data structures exchanged between the Swagger parser, JAR parser, and configuration generator enable each utility to be extended or replaced independently.

The platform’s extensibility is further supported by its clean separation of concerns: authentication middleware can be introduced at the Express layer without modifying route logic, CORS policies can be tightened through configuration changes, and the `sql.js` persistence layer can be upgraded to a native SQLite binding for improved throughput — all without requiring architectural redesign. These characteristics position the platform well for incremental evolution as organisational requirements grow, as discussed in the roadmap presented in the following chapter.

5.11 Performance Observations

The platform demonstrates acceptable performance characteristics for its intended single-user or small-team usage pattern. Swagger specification parsing completes within sub-second timeframes for typical specification files containing up to several hundred endpoints, with the reference resolution cache ensuring that repeated `$ref` lookups within the same specification are resolved in constant time after the initial traversal.

JAR bytecode parsing constitutes the most computationally intensive operation, particularly for large archives containing hundreds of class files with deep type hierarchies. The browser-side parser implements cooperative yielding through batched processing of twenty-four class files per cycle, preventing UI thread blocking during parsing. The server-side parser, operating without this constraint, applies per-flatten path caps of three hundred entries to bound memory consumption. A flatten cache keyed by type name, prefix, and depth limit prevents redundant traversal of identical type hierarchies across methods, yielding meaningful speedups for JAR files containing many methods with overlapping parameter types.

Module generation duration is dominated by the external Maven build process and PowerShell script execution, typically ranging from thirty seconds to several minutes depending on module

complexity and host machine resources. The real-time output streaming mitigates the perceived impact of longer generation times by providing continuous progress visibility rather than a silent wait. A ten-minute spawn timeout provides a safety bound against processes that hang indefinitely, with graceful termination and user notification upon timeout expiry.

CHAPTER 6

Conclusion and Future Work

6.1 Conclusion

This thesis has presented a comprehensive technical study of an API connector development platform engineered to streamline the integration workflow within an enterprise telecom environment. Through systematic source-code-level analysis and architectural evaluation, the investigation documented the platform’s design decisions, implementation patterns, operational capabilities, and the areas in which further evolution is warranted.

At the heart of the platform’s contribution is its successful consolidation of a complete end-to-end connector development workflow. Five formerly disconnected phases — module management, specification parsing, bytecode-driven EJB metadata extraction, interactive field mapping with configuration generation, and automated module generation — are integrated into a unified, tool-assisted process. Where engineers previously navigated between specification analysis tools, bytecode decompilation utilities, configuration editors, and command-line build scripts in an ad-hoc sequence, the platform provides a structured path from initial specification upload to deployable module output, reducing onboarding effort, enforcing structural consistency across generated configurations, and establishing an auditable trail of mapping decisions.

The React frontend realises this vision through an intuitive, guided interface that presents the connector development process as a structured sequence of steps, lowering the expertise

barrier for engineers unfamiliar with the target enterprise services. On the server side, the Node.js/Express backend demonstrates strong orchestration capabilities, managing file storage, database persistence, filesystem synchronisation, and external process execution through a well-organised route structure. Underpinning both layers, the utility tier — comprising the Swagger parser, JAR introspection engine, and configuration generator — provides the algorithmic foundation that enables automated metadata extraction and mapping from both API specifications and compiled enterprise artifacts.

Real-time generation feedback, implemented through Socket.IO event streaming, transforms module generation from an opaque batch operation into a transparent, observable process. The hybrid synchronisation strategy that combines chokidar filesystem watchers, Socket.IO event broadcasting, and periodic polling fallback ensures data consistency across the platform's distributed state while accommodating the connectivity constraints characteristic of enterprise network environments.

Beyond its current capabilities, the evaluation identifies clear opportunities for progressive enhancement. Authentication and authorisation middleware can be introduced at the Express layer to extend the platform beyond trusted internal networks. The storage layer is architecturally prepared for migration to a native SQLite binding for improved throughput. The generation pipeline can be expanded with cross-platform support through a Node.js-based orchestrator. Automated testing infrastructure can be layered onto the existing modular codebase. These opportunities reflect the natural evolution path of a well-architected platform and are readily achievable within the existing design framework.

6.2 Key Contributions

This thesis makes several contributions to the documentation and understanding of the API connector development platform.

First, it provides complete architectural documentation of the two-tier system design, encompassing the frontend workflow engine, backend orchestration layer, utility algorithms, and filesystem-based storage model. Second, it delivers a detailed analysis of the Swagger parsing algorithm — including reference resolution with `WeakMap`-based caching, recursive schema flattening with circular reference detection, and operation detail extraction with parameter merging — demonstrating how API specifications are transformed into structured metadata suitable for downstream consumption.

Third, the thesis documents the JAR bytecode introspection pipeline in depth, covering JVM descriptor parsing, generic signature resolution, EJB interface detection through annotation and heuristic strategies, deep field path extraction through recursive type traversal with configurable bounds, and synthetic parameter name normalisation. This documentation provides a technical reference for the platform’s most distinctive capability: extracting comprehensive service metadata from compiled enterprise artifacts without access to the original source code.

Fourth, the evaluation yields a structured assessment spanning security, reliability, portability, and maintainability dimensions, with each finding supported by specific evidence from the codebase. The consolidated findings matrix provides a concise summary suitable for both technical stakeholders and academic audiences. Finally, the prioritised evolution roadmap furnishes a clear understanding of the platform’s maturity profile and the opportunities available for future enhancement.

6.3 Future Work

Although the platform delivers substantial value in its current form, several areas of enhancement would materially improve its production readiness, operational resilience, and broader applicability. Table 4 provides an overview of the recommended enhancements with their priority levels and estimated effort. The subsections that follow elaborate on each area.

Table 4: Prioritised future work roadmap.

Enhancement Area	Priority	Effort	Key Deliverable
Security enhancement	High	Medium	JWT authentication, RBAC, input sanitisation
Cross-platform generation	High	High	Node.js-based generation orchestrator
Storage evolution	High	Medium	Native SQLite or PostgreSQL migration
Testing infrastructure	High	High	Unit, integration, and E2E test suites
Frontend refactoring	Medium	Medium	Component decomposition with React hooks
Containerised deployment	Medium	Low	Docker image with Kubernetes support

6.3.1 Security Hardening

A natural extension of the platform’s capabilities is the addition of authentication and authorisation mechanisms. Implementing token-based authentication using JSON Web Tokens would enable identity verification for API requests, while role-based access control would govern sensitive operations such as module deletion and generation execution. Enhanced input validation and sanitisation across all API endpoints would further strengthen the platform’s security posture. Adopting externally signed TLS certificates alongside the existing self-signed capability would enable seamless browser trust. Fine-grained CORS configuration would provide origin-level access control for multi-domain deployments.

6.3.2 Cross-Platform Generation Support

Extending the generation pipeline with a cross-platform execution strategy would broaden the platform’s deployment reach. One approach involves creating a Node.js-based generation orchestrator that replaces the PowerShell script with equivalent JavaScript logic for directory

scaffolding, template expansion, and Maven invocation, using Node.js child process management for external command execution. An alternative retains the existing scripts but implements an OS-aware runner that selects PowerShell on Windows and a Bash-compatible equivalent on Linux. Either path should proceed incrementally, maintaining parallel support for both execution backends during the transition period to avoid disrupting existing workflows.

6.3.3 Storage and Data Management Evolution

Upgrading the storage layer from `sql.js` to a native SQLite binding such as `better-sqlite3` — or introducing PostgreSQL for larger deployments — would deliver transactional write guarantees, concurrent access support, and further improved query performance. Under this model, the filesystem would continue to serve as the primary repository for specification files, configuration YAML, and generated artifacts, while the database would assume responsibility for relational metadata, deduplication records, and operational state. This hybrid approach preserves the auditability benefits of filesystem-based storage while adding the reliability characteristics needed for multi-user scenarios.

6.3.4 Testing Infrastructure

Building out an automated testing suite should follow a progressive strategy. Unit tests for the Swagger parser, JAR parser, and configuration generator utilities would protect the platform’s most algorithmically complex components, ensuring continued accuracy of mappings and configurations as the codebase evolves. Integration tests for the backend API endpoints would verify end-to-end request processing, database operations, and file management. End-to-end tests using browser automation would validate the complete workflow from specification upload through module generation. Incorporating these tests into a continuous integration pipeline would ensure automatic execution on every code change, establishing a comprehensive quality assurance framework.

6.3.5 Frontend Architectural Improvement

Evolving the `YamlUploader` component into smaller, focused subcomponents would further enhance maintainability as the platform grows. A state machine or reducer-based architecture would formalise workflow transitions and make the component’s behaviour more predictable and testable. Extracting domain-specific logic into custom React hooks — for module management, specification handling, JAR processing, mapping state, and generation orchestration — would improve code organisation and enable independent testing of each workflow concern.

6.3.6 Containerised Deployment

Docker-based containerisation would simplify environment setup, improve deployment reproducibility, and enable orchestration through Kubernetes for scaling and high availability. A multi-stage Docker build could package the frontend build artifacts, backend application, and required runtime dependencies into a single deployable image. Container orchestration would, in turn, enable horizontal scaling, automated health monitoring with restart policies, and rolling update deployments — extending the platform’s operational capabilities to large-scale enterprise environments.

6.4 Scalability Considerations

The platform’s modular architecture provides a sound foundation for both functional and organisational scaling. The backend’s route-based organisation allows new functional domains to be introduced as independent route modules. The frontend’s component architecture supports additional workflow steps through independent component development. The specification parser’s normalised intermediate representation can be extended to accommodate specification formats beyond OpenAPI and Swagger.

From an organisational perspective, the system could be enhanced with multi-tenant isolation using project-level access control, enabling multiple teams to share a single deployment while maintaining strict data separation. A queue-based architecture for generation operations would permit parallel processing of multiple concurrent generation requests, improving throughput for organisations managing large specification portfolios. Integration with enterprise identity providers through SAML or OAuth 2.0 would support seamless onboarding within organisations that mandate centralised authentication.

6.5 Practical Applications

The platform holds significant potential for adoption beyond its immediate deployment context. Integration engineering teams in telecom, financial services, and healthcare sectors face comparable challenges in mapping API specifications to enterprise service interfaces, and the workflow-driven approach demonstrated by this platform is readily transferable across these domains. The bytecode introspection capability is particularly portable, as it addresses a common need to understand compiled service artifacts in environments where source code access is unavailable or impractical.

Beyond its operational utility, the guided workflow interface also serves a training and onboarding function, reducing the expertise threshold required for connector development and enabling junior engineers to produce consistent, enterprise-compliant configurations. Furthermore, the specification parsing and field mapping utilities could be extracted as standalone libraries for integration into other development tools, providing reusable building blocks for API-driven code generation workflows.

6.6 Summary

In summation, this thesis demonstrates that a workflow-driven, tool-assisted approach to API connector development is both technically viable and practically valuable for enterprise integration teams. The platform converts a fragmented manual process into a guided, repeatable, and transparent workflow, delivering measurable improvements in consistency, traceability, and developer experience. The technical evaluation confirms strong capabilities across specification parsing, bytecode introspection, real-time communication, persistence management, and deployment flexibility.

The phased enhancement roadmap proposed in this thesis charts a clear path for extending the platform's capabilities into broader enterprise contexts. Each recommended enhancement builds naturally upon the existing architecture and is prioritised to balance immediate operational value with longer-term capability growth. The platform's modular architecture and clean separation of concerns furnish a solid foundation for this evolutionary journey, ensuring that future enhancements can be pursued incrementally without requiring fundamental redesign.

References

- [1] Apache Software Foundation. Apache maven project documentation. <https://maven.apache.org/guides/>, 2025. Accessed: April 2026.
- [2] Apache Software Foundation. Apache tomcat 9 documentation. <https://tomcat.apache.org/tomcat-9.0-doc/>, 2025. Accessed: April 2026.
- [3] Chokidar Contributors. Chokidar file watching library. <https://github.com/paulmillr/chokidar>, 2025. Accessed: April 2026.
- [4] Express.js Contributors. Express.js documentation. <https://expressjs.com/>, 2025. Accessed: April 2026.
- [5] Meta Open Source. React documentation. <https://react.dev/>, 2025. Accessed: April 2026.
- [6] Multer Contributors. Multer middleware for node.js. <https://github.com/expressjs/multer>, 2025. Accessed: April 2026.
- [7] OpenAPI Initiative. Openapi specification. <https://spec.openapis.org/oas/latest.html>, 2025. Accessed: April 2026.
- [8] Socket.IO Contributors. Socket.io documentation v4. <https://socket.io/docs/v4/>, 2025. Accessed: April 2026.
- [9] sql.js Contributors. sql.js documentation. <https://sql.js.org/>, 2025. Accessed: April 2026.

APPENDIX A

Biographical Sketch

Mainak Dasgupta

AD Nagar, Agartala, Tripura, PIN-799003,

E-Mail: dasguptamainak02@gmail.com, Contact. No. +91-9366415050

- Pursuing B.Tech. in Computer Sc. & Engg. branch from NIT Agartala with CGPA of 7.27.
- High School from Umakanta Academy (English Medium) under (T.B.S.E), Tripura with 87 % in 2019.
- Intermediate from Pranavanada Vidyamandir, under (C.B.S.E), Tripura with 77 % in 2021.